

Chiamate di sistema per i processi

Lezione 12

Sistemi Operativi - Il Modulo
(Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- La gerarchia dei processi
- System calls e funzioni di libreria per la gestione dei processi

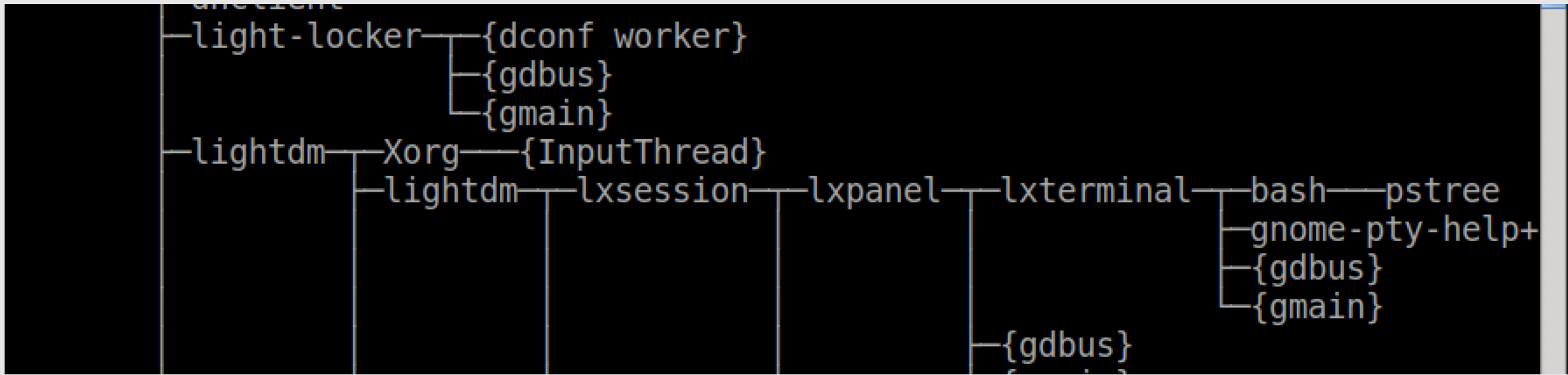


La creazione: `fork`

- `init` è il processo (pid = 1) genitore di tutti i processi del sistema in esecuzione
- da esso vengono creati tutti i processi mediante system call `fork()`
- la creazione consiste nella duplicazione del processo chiamante e creare relazioni **genitore -- figlio**
- con `ps tree 1` possiamo visualizzare l'albero (genealogico) dei processi

```
studente@debian9:~$ ps tree 1
systemd -- (gmain)
├── ModemManager -- (gdbus)
│   └── (gmain)
│       ├── VBoxClient -- (SHCLIP)
│       ├── VBoxClient -- (X11 events)
│       ├── VBoxClient -- (dndHGCN)
│       └── VBoxClient -- (dndX11)
│           └── VBoxService -- {automount}
│               ├── {control}
│               ├── {cpuhotplug}
│               ├── {memballoon}
│               ├── {timesync}
│               ├── {vminfo}
│               └── {vmstats}
├── acpid
├──agetty
├── applet.py -- (gmain)
├── at-spi2-registr -- (gdbus)
│   └── (gmain)
├── clipit -- (gdbus)
│   └── (gmain)
├── cron
├── dbus-daemon
├── dhclient
├── light-locker -- (dconf worker)
│   ├── (gdbus)
│   └── (gmain)
├── lightdm
│   ├── Xorg -- {InputThread}
│   └── lxsession
│       ├── lxpanel
│       ├── lxterminal
│       ├── bash
│       ├── firefox-esr
│       └── Web Content -- {BgHangManager}
│           ├── {Chrome_ChildThr}
│           ├── {Hang Monitor}
│           ├── {ImageBridgeChil}
│           ├── {ImageIO}
│           ├── {ImgDecoder #1}
│           ├── 5*[{JS Helper}]
│           ├── {JS Watchdog}
│           ├── {ProcessHangMoni}
│           ├── {Socket Thread}
│           ├── {Timer}
│           ├── {VideoChild}
│           ├── {gdbus}
│           └── {gmain}
│               ├── {BgHangManager}
│               ├── {Cache I/O}
│               ├── {Cache2 I/O}
│               ├── {Compositor}
│               ├── {DNS Resolver #1}
│               ├── {DNS Resolver #2}
│               ├── 3*[{DOM Worker}]
│               ├── 3*[{DataStorage}]
│               ├── {GMPThread}
│               ├── {Gecko IOThread}
│               ├── {HTML5 Parser}
│               ├── {Hang Monitor}
│               ├── {IPDL Background}
│               ├── {ImageBridgeChil}
│               ├── {ImageIO}
│               ├── {ImgDecoder #1}
│               ├── {IndexedDB #3}
│               ├── 5*[{JS Helper}]
│               ├── {JS Watchdog}
│               ├── {Link Monitor}
│               ├── {ProcessHangMoni}
│               ├── {Proxy Resolution}
│               ├── {Socket Thread}
│               ├── {SoftwareVsyncTh}
│               ├── {Storage I/O}
│               ├── {StreamTrans #15}
│               ├── {Timer}
│               ├── {URL Classifier}
│               ├── {dconf worker}
│               ├── {firefox-esr}
│               ├── {gdbus}
│               ├── {gmain}
│               ├── {localStorage DB}
│               ├── {mozStorage #10}
│               ├── {mozStorage #1}
│               ├── {mozStorage #2}
│               ├── {mozStorage #3}
│               └── {mozStorage #4}
```

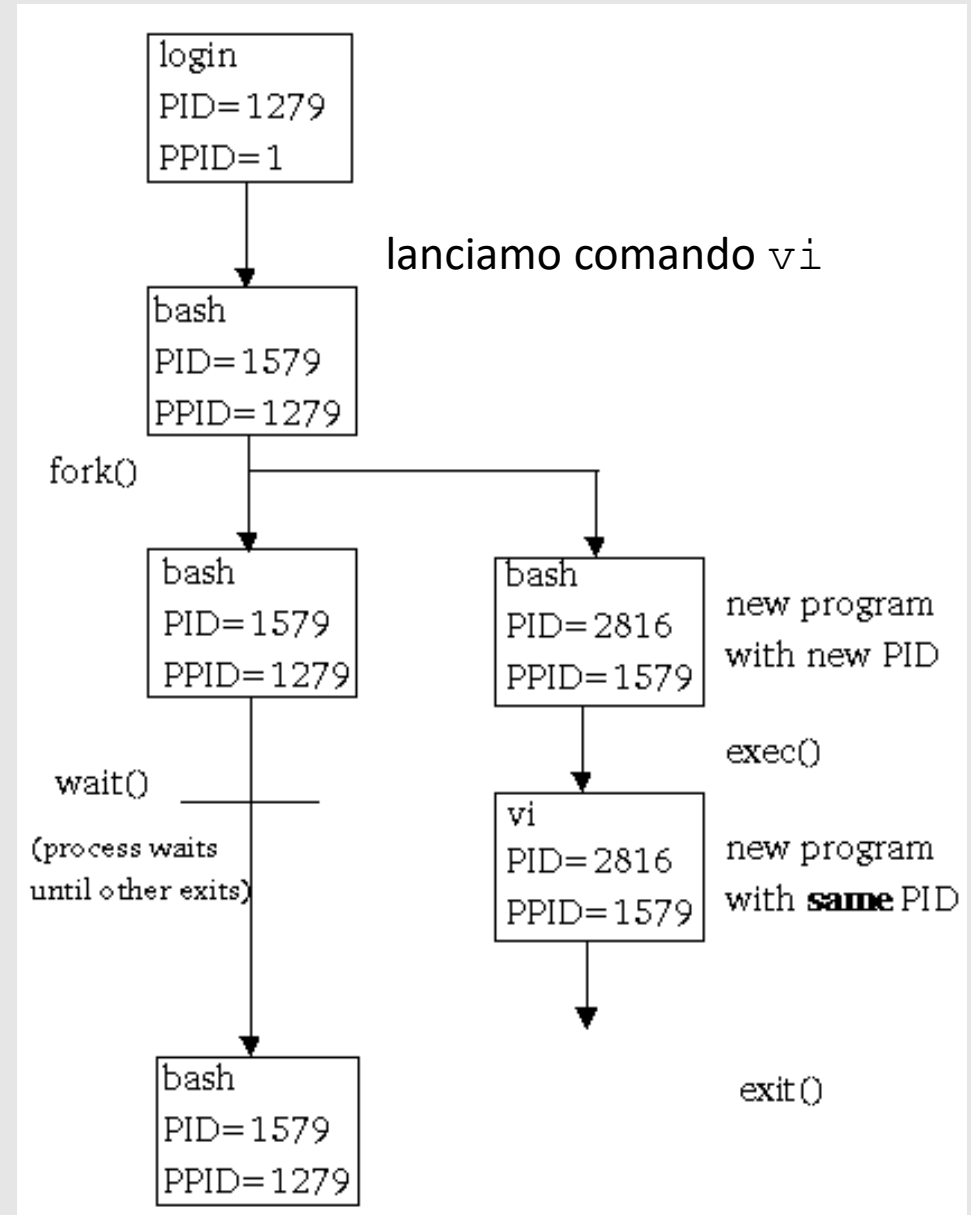
- e vediamo che la nostra `bash` è figlia di `lxterminal` e il `pstree` che stiamo usando è figlio di `bash`



- ESERCIZIO: provate dalla vostra `bash` lanciate in background `gedit` e `firefox` e poi visualizzate l'albero «locale» con `pstree`

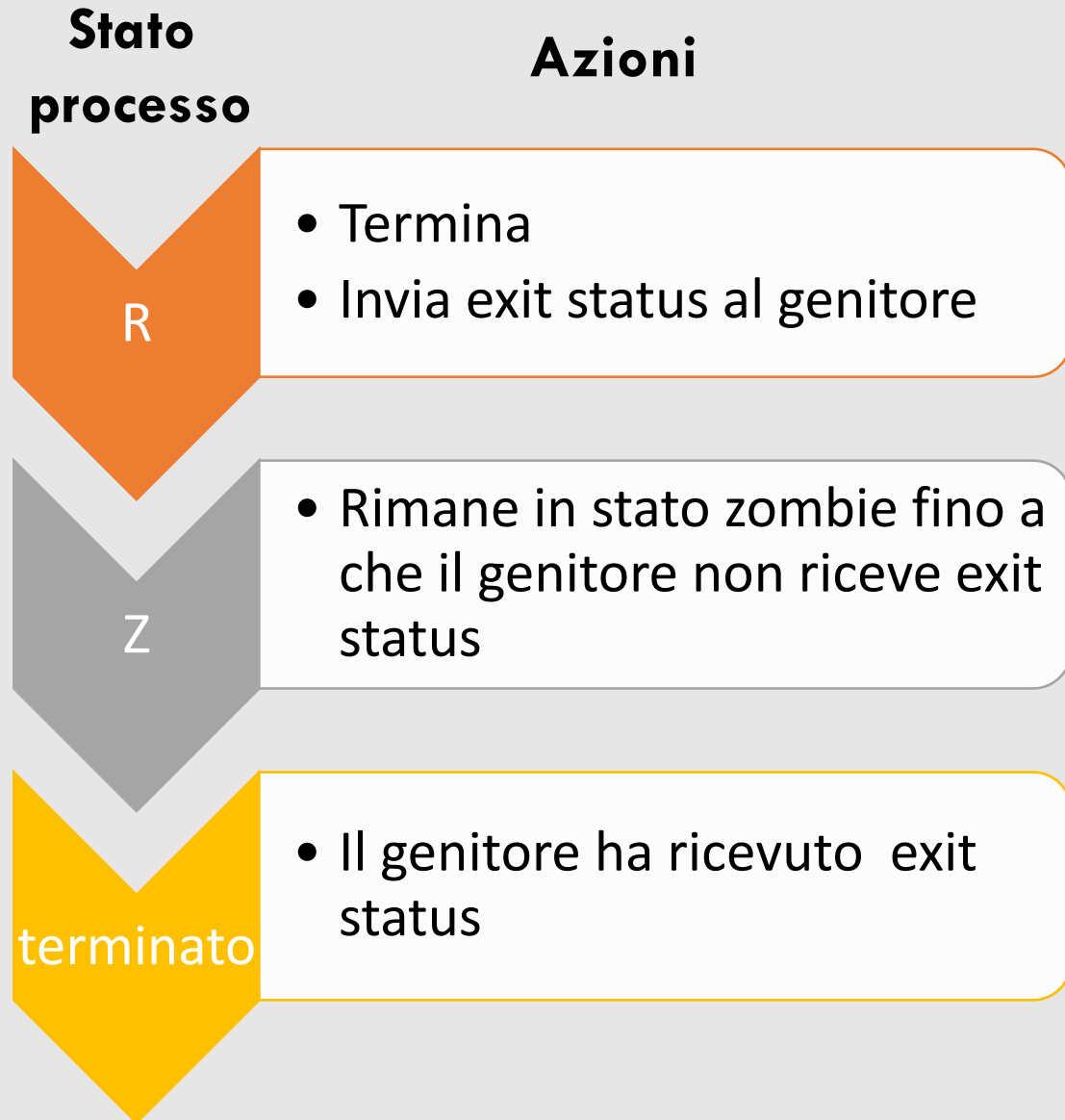
Nascita e morte (di processi)

- Ogni processo nella gerarchia fa riferimento al processo genitore, ovvero
 - quando nasce eredita codice e parte dello stato dal genitore
 - quando termina/muore ritorna l'*exit status* (codice di uscita) al genitore
- In caso il genitore termini/muoia prima del processo figlio, quest'ultimo è adottato da init



Zombie

- Un processo attraversa le fasi a lato
 - R=running, Z=zombie
- Un processo Zombie è quindi un processo terminato, ma il cui Process Control Block è mantenuto nella Process Table dal kernel per dare modo al processo genitore di leggere l'exit status di tale processo.
- Se il genitore di un processo muore/termina il processo rimane in stato Zombie



System calls controllo processi

attributi	creazione/terminazione	sincronizzazione	esecuzione	ambiente
getpid getppid get/setuid get/setgid	fork exit abort (funzione)	wait waitpid	execv*	getenv setenv putenv unsetenv clearenv

Syscall	Azione	Risultato
<code>pid_t getpid(void);</code>	Ritorna pid processo chiamante	Terminano sempre con successo
<code>pid_t getppid(void);</code>	Ritorna pid genitore del processo chiamante	
<code>uid_t getuid(void);</code>	Ritorna uid processo chiamante	
<code>uid_t geteuid(void);</code>	Ritorna effective uid processo chiamante	
<code>int setuid(uid_t uid);</code>	Imposta effective uid processo chiamante a uid (parametro input);	ritornano -1 in caso di errore, 0 in caso di successo
<code>int setgid(uid_t uid);</code>	Imposta effective gid processo chiamante a gid (parametro input);	

Richiedono

```
#include <sys/types.h>
#include <unistd.h>
```

Per dettagli vedi

```
man 2 <syscall_name>
```



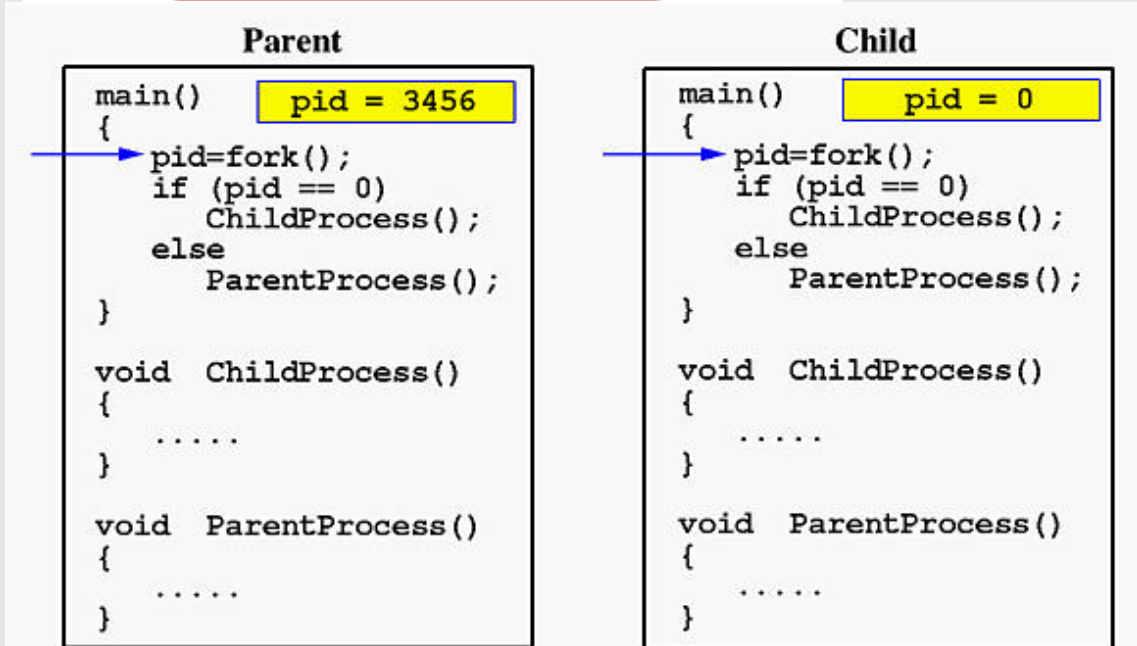
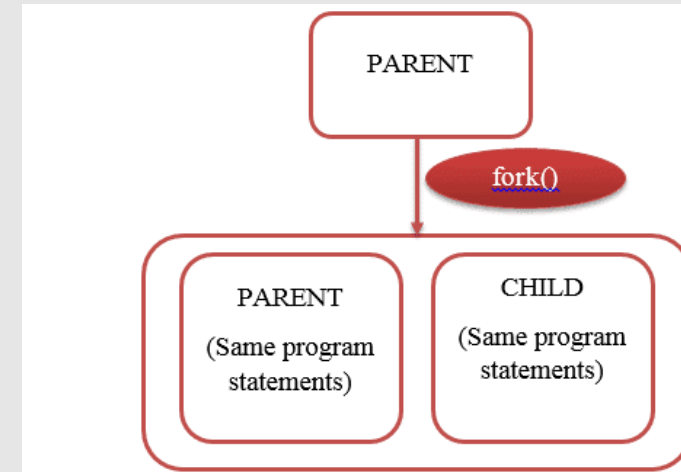
Esempi ed esercizi

- Vedere gli esempi:
 - `getgid.c`
 - `getppid.c`
 - `getuid.c`
 - `setgid.c`
 - `setuid.c`
- Nei file `.c` ci sono anche esercizi da fare



pid_t fork(void);

- Crea un nuovo processo che è la copia del processo chiamante, a parte alcune strutture dati, come ad esempio il pid
- Una chiamata `fork()` viene eseguita una volta sola, ma ritorna due volte
 - una volta al processo che l'ha invocata
 - una volta ritorna al nuovo processo che è stato generato dall'esecuzione della `fork` stessa.
- In caso di errore ritorna -1 al chiamante e non viene creato nessun processo figlio



Ereditarietà attributi (processo figlio)

NON ereditati

- process id (pid)- Il figlio ha il suo proprio pid
- parent pid (ppid) - Nel figlio il parent pid è uguale al pid del genitore
- i timer - Ogni processo ha i propri timer
- record lock / memory lock - Due processi non possono detenere gli stessi lock
- contatori risorse - I contatori dell'utilizzo delle risorse sono impostati a zero nel figlio
- coda dei segnali - La coda dei segnali in attesa viene svuotata nel figlio

Ereditati

- real ed effective user e group ID
- groups id
- working directory
- ambiente del processo
- descrittori dei file
- terminale di controllo
- memoria condivisa



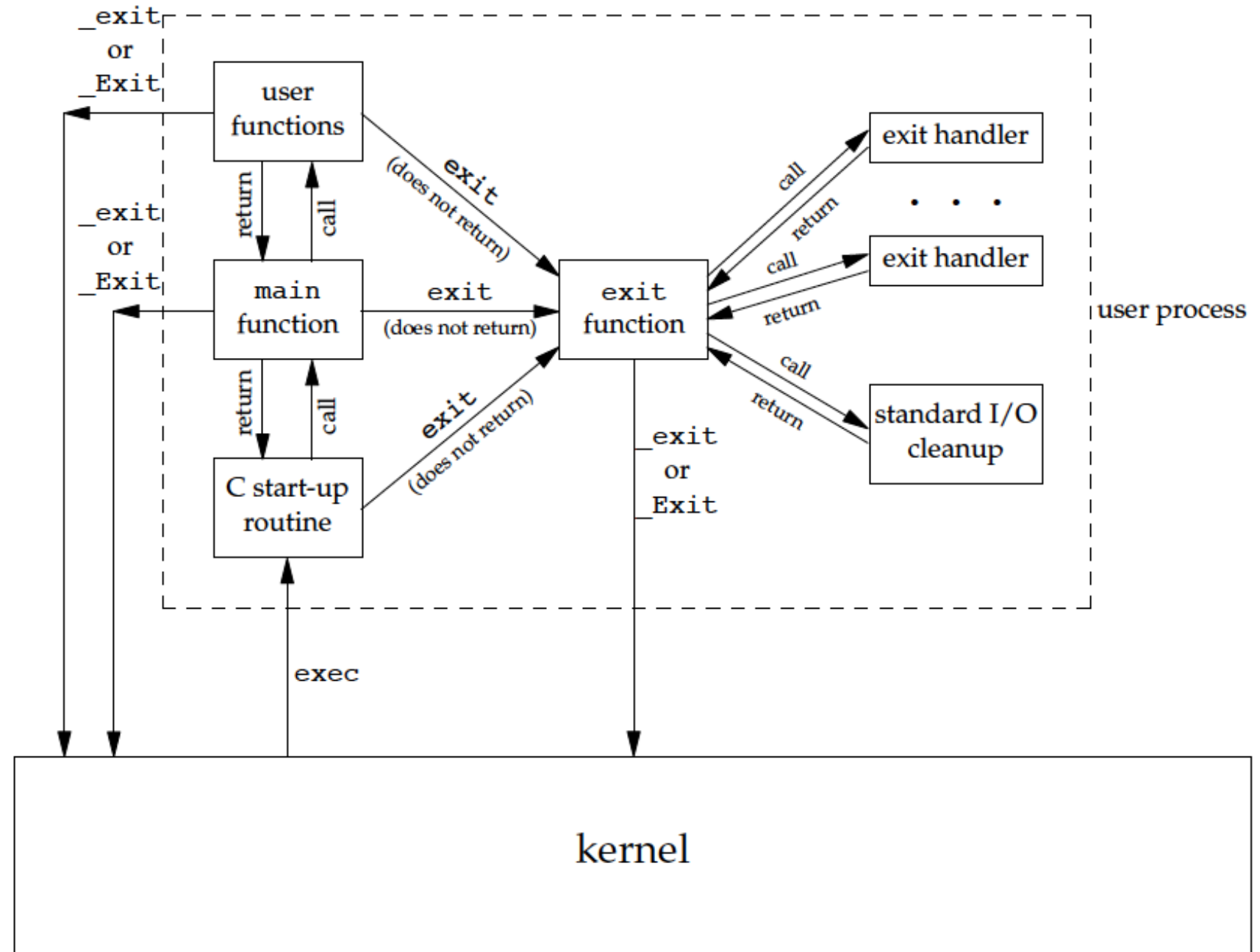
```
void _exit(int status);    (#include <unistd.h>)
```

```
void exit(int status);    (#include <stdlib.h>)
```



- `_exit` (syscall)
 - Termina direttamente il processo che la invoca senza invocare *handler* (gestore)
 - Chiude tutti i file descriptor
 - I figli vengono ereditati dal processo 1 (`init`)
 - Invia il segnale SIGCHLD al processo genitore
 - Ritorna `status` e l'`exit status` al processo genitore
- `exit` (funzione libreria)
 - Invoca tutti gli handler registrati con `atexit` e `on_exit`
 - Chiude tutti i file descriptor, svuota gli stream `stdio` e li chiude
 - Termina il processo
 - Ritorna (`status & 0377`) al genitore (vedi `wait()`)
 - `EXIT_SUCCESS` e `EXIT_FAILURE` sono 2 costanti predefinite che possono essere passate come `status` (soluzione portabile)

Lancio e terminazione di un programma C



Esercizi

- Aprite `exit.c` e fate l'esercizio
- Aprite `fork2.c` e modificate il codice in modo che:
 - apra il file passato sulla linea di comando in modalità scrittura e lettura
 - le due `printf` alla fine avvengano su tale file



```
void abort(void) ;    (#include stdlib.h)
```



- Invia SIGABRT al processo chiamante: il processo termina in modo anormale
- La SIGABRT può essere intercettata e gestita (ma poi il processo verrà terminato comunque)
- [Fino alla glibc 2.26, `abort` chiudeva e «svuotava» (*flush*) tutti i file. POSIX permette (ma non richiede!) tale comportamento da `abort`. Vedi `man abort`.]
- vedi `abort.c`

wait() , waitpid()

```
#include <sys/types.h>  
#include <sys/wait.h>
```



- Queste chiamate di sistema si usano per
 - attendere cambiamenti di stato in un figlio del processo chiamante, e
 - ottenere informazioni sul figlio il cui stato è cambiato
- Un cambiamento di stato avviene quando
 - il processo figlio è terminato
 - il figlio è stato arrestato da un segnale
 - il figlio è stato ripristinato da un segnale
- Se un figlio è terminato
 - wait/waitpid permettono al sistema di rilasciare le risorse associate al figlio;
 - se non viene eseguita un'attesa, allora il figlio terminato rimane in uno stato "zombie"
- Se un figlio ha già cambiato stato, allora le chiamate ritornano immediatamente
- Altrimenti esse si bloccano fino a quando un figlio cambia stato o un gestore di segnale interrompe la chiamata


```
pid_t wait(int *status);
```

- La chiamata di sistema `wait()` sospende l'esecuzione del processo chiamante fino a quando uno dei suoi figli termina
- ritorna -1 in caso di errore, altrimenti il pid del figlio terminato
- `status` punta ad un intero contenente informazioni di stato (vedi `man`)

```
pid_t waitpid(pid_t pid, int
*status, int options);
```

- Sospende l'esecuzione del processo chiamante fino a quando un figlio specificato dall'argomento `pid` ha cambiato stato
- Il valore di `pid` può essere:
 - `< -1` : attesa di qualunque processo figlio il cui gruppo ID del processo sia uguale al valore assoluto di `pid`
 - `-1` : aspetta qualunque processo figlio
 - `0` : aspetta qualunque processo figlio il cui gruppo ID del processo sia uguale a quello del processo chiamante
 - `> 0` : aspetta il figlio il cui ID di processo sia esattamente `pid`

```
pid_t waitpid(... int options);  
(cont.)
```

- Il comportamento predefinito di `waitpid()` è attendere solo i figli terminati
- Questo comportamento è modificabile attraverso `options`
 - `WNOHANG` ritorna immediatamente se nessun figlio è uscito (`exit`).
 - `WUNTRACED` ritorna anche se un figlio si è arrestato (ma non tracciato attraverso `ptrace(2)`). Lo stato del figlio non tracciato che è stato arrestato è fornito anche se l'opzione non è specificata.
 - `WCONTINUED` (A partire da Linux 2.6.10) ritorna anche se un figlio arrestato è stato riesumato inviando `SIGCONT`
- i valori W^* possono essere messi in OR.

Verifica dello stato ritornato alla `wait/waitpid`

```
int s;  
wait(&s);  
WIFEXITED(s); //o altra macro W*
```

- `wait` memorizza in `status` (se non NULL)
 - il valore dello stato del processo figlio
- `status` può essere verificato con le seguenti macro
 - `WIFEXITED` ritorna true se terminato normalmente
 - `WEXITSTATUS` ritorna l'exit status del figlio
 - `WIFSIGNALED` ritorna true se è terminato per la ricezione di un segnale
 - `WTERMSIG` ritorna il numero del segnale che ha causato la terminazione
 - `WCOREDUMP` ritorna true se ha generato un core dump
 - `WIFSTOPPED` ritorna true se il processo è in stato stopped per la ricezione di un segnale
 - `WSTOPSIG` ritorna il numero del segnale che ha causato lo switch in stato di stopped
 - `WIFCONTINUED` ritorna true se il processo ha continuato per la ricezione di un segnale `SIGCONT`

- wait.c
- wait2.c
- waitpid.c

```

studente@debian9: ~/Lezione12/Lezione12
File Modifica Schede Aiuto
First child's output, value = 10
First child (pid = 1714) is about to exit
*** Parent detects process 1714 is done ***
*** Parent exits ***
studente@debian9:~/Lezione12/Lezione12$ ./wait.out
*** Parent is about to fork process 1 ***
*** Parent is about to fork process 2 ***
*** Parent enters waiting status .....
Second child process starts (pid = 1720)
Second child's output, value = 1
Second child's output, value = 2
Second child's output, value = 3
Second child's output, value = 4
Second child's output, value = 5
Second child's output, value = 6
First child process starts (pid = 1719)
First child's output, value = 1
First child's output, value = 2
First child's output, value = 3
First child's output, value = 4
First child's output, value = 5
First child's output, value = 6
First child's output, value = 7
First child's output, value = 8
First child's output, value = 9
First child's output, value = 10
First child (pid = 1719) is about to exit
*** Parent detects process 1719 was done ***
Second child's output, value = 7
Second child's output, value = 8
Second child's output, value = 9
Second child's output, value = 10
Second child (pid = 1720) is about to exit
*** Parent detects process 1720 is done ***
*** Parent exits ***
studente@debian9:~/Lezione12/Lezione12$

```



Esercizio

- Riprendete l'esercizio su `fork2.c` e fate in modo che il processo figlio scriva per primo sul file.

Come rimpiazzare un processo figlio con un altro processo

- `fork()` crea 2 processi identici con attributi comuni e altri no
- posso usare il controllo di flusso per far sì che il processo figlio faccia qualcosa di diverso dal genitore ... ma poco efficiente
- posso usare invece la syscall `execve` o le funzioni della famiglia `exec`
 - sostituiscono l'immagine del processo che la invoca con una nuova immagine

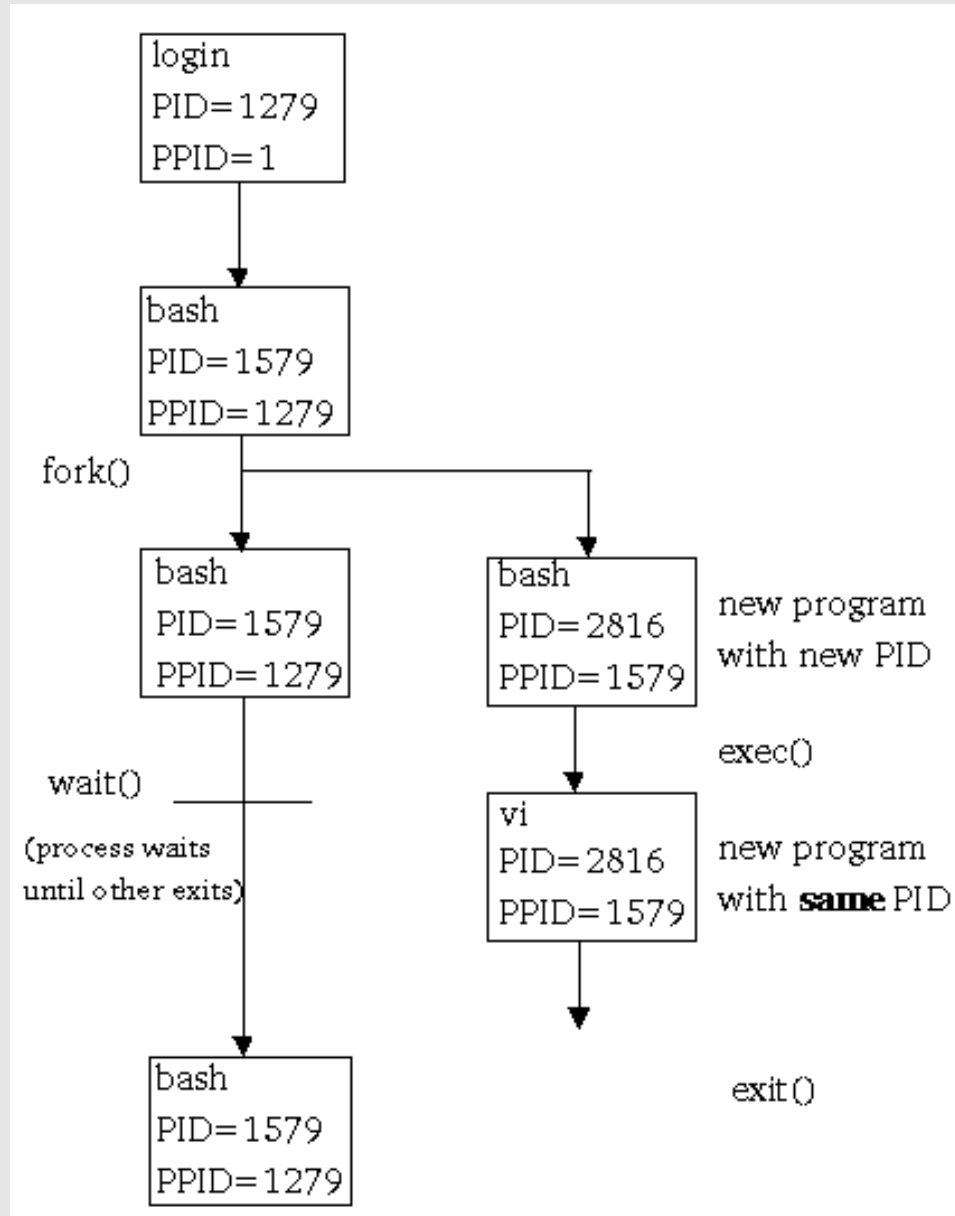
```
int execve(const char *filename, char *const argv[], char *const envp[]);
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execl_e(const char *path, const char *arg, ..., char * const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

funzioni
di libreria



Esempio

- `exec` esegue il programma `vi`




```
int execve(const char *filename, char *const
argv[], char *const envp[])
```

- sostituisce l'immagine del processo con quella contenuta in `filename`, il quale può essere:
 - Un file binario (cioè un eseguibile), oppure
 - Uno script che inizia con `#! interpreter [optional-arg]`
- `argv[]` contiene gli argomenti in input per il comando eseguito; `argv[0]` deve essere il comando stesso; l'array deve essere terminato con `NULL`
- `envp[]` contiene l'ambiente della nuova immagine (dopo)
- `execve` sostituisce le zone di memoria **text**, **bss**, e **stack** del processo che la invoca con quelle del nuovo programma mandato in esecuzione
- `execve` ritorna -1 se errore, altrimenti non ritorna (ovviamente!)

execve e gli attributi di processo

Mantenuti

- process ID (PID) e parent PID
- real uid e real gid
- groups id
- session ID
- terminale di controllo
- working directory e root directory
- maschera creazione file (umask)
- file locks
- file descriptors (ad eccezione di quelli con flag FD_CLOEXEC impostato)
- maschera dei segnali
- segnali in attesa

Non mantenuti

- effective uid ed effective gid.
- memory mapping
- timers
- memoria condivisa
- memory lock



Le funzioni di libreria `exec*`

- `execl`, `execvp`, `execle`
 - prendono in ingresso un numero variabile di puntatori per specificare gli argomenti in ingresso alla nuova immagine del processo; un puntatore per ogni argomento
- `execv`, `execvp`, `execvpe`
 - prendono in ingresso un puntatore ad un vettore per specificare gli argomenti in ingresso alla nuova immagine; un vettore dove ogni elemento è un argomento in ingresso.

```
int execl(const char *path, const char *arg, ...);
int execvp(const char *file, const char *arg, ...);
int execle(const char *path, const char *arg, ..., char * const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

Le funzioni di libreria `exec*` (cont.)

- `execlp`, `execvp` ed `execvpe`
 - nel caso il nome del file non contenga uno slash (/), usano la variabile d'ambiente `PATH` per cercare il file da utilizzare per la nuova immagine
 - le altre usano path relativo o assoluto specificato in `filename`
- `execle`, `execvpe`
 - il parametro `envp` consente di specificare l'environment (l'ambiente) della nuova immagine

```
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execle(const char *path, const char *arg, ..., char * const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

Ambiente di un processo

- E' costituito da una serie di stringhe della forma *key=value*. Di solito *key* è in maiuscolo, ad es.:

```
PATH=/home/pz/.local/bin:/home/pz/bin:/usr/local/bin:/usr/local/sbin:/usr/bin:/usr/sbin
```

- E' definito tramite un array di puntatori a caratteri, terminato da NULL
- Per default, l'ambiente di un processo coincide con l'ambiente del processo genitore (ereditato mediante `fork`)
- Permettono di cambiare l'ambiente mediante parametro `envp[]`

```
int execl(const char *filename, char *const argv[], char *const envp[]);  
int execl(const char *path, const char *arg, ..., char *const envp[]);  
int execvp(const char *file, char *const argv[], char *const envp[]);
```

Modifica ambiente

- Per poter accedere all'ambiente da un programma C si può:

- Aggiungere l'argomento `envp` ai parametri del main

```
main(int argc, char *argv[], char *envp[])
```

- Non è POSIX compatibile, quindi poco portabile

- Potete invece utilizzare la variabile globale `**environ`

```
extern char **environ;
```

Funzioni utili per la gestione dell'ambiente

`(#include <stdlib.h>)`

- `char *getenv(const char *name);`
 - ritorna il valore nella variabile `name`
- `int setenv(const char *name, const char *value, int overwrite);`
 - imposta (o aggiunge la variabile) `name = value`
- `int putenv(char *string);`
 - come sopra `string="name=value"`
- `int unsetenv(const char *name);`
 - rimuove `name` dalle variabili di ambiente
- `int clearenv(void);`
 - «ripulisce» l'ambiente e imposta `environ=NULL`

Esempi

- `getenv.c`
- `print_env.c`
- `setenv.c`
- `execve.c`



Cambiare working dir o root dir del nuovo processo

```
#include <unistd.h>
```

```
int chdir(const char *path);
```

```
int chroot(const char *path);
```

Domande?

